PLAY STORE APPLICATION

Capstone Project Report

Submitted in fulfilment of the requirements for

MERN Stack Capstone Project | Great Learning

Submitted by

Galeti Mohammad Hussain

hussaingaleti786@gmail.com

March 2026

Instructor & Guide:

Mr.Abhishek sharma

TABLE OF CONTENTS: page no:

1. Introduction....3
2. Problem Statement....4
3. Need for the Project....5
4. Project Objectives.... 6-7
5. Scope of the Project....8
6. Stakeholders and User Roles....9
7. Requirement Analysis.... 10-11
8. Development Methodology and Sprint Execution.... 12-13
9. Technology Stack and Justification.... 13-15
10. System Architecture.... 16-17
11. Entity Relationship Diagram.... 18
12. Project Folder Structure....19
13. Frontend Design and Module Structure.... 20
14. Backend Design and Module Structure.... 21
15. Database Design and Relationship Model.... 21
16. Detailed Collection Design.... 21-22
17. API Design and Endpoint Summary.... 22
18. Authentication, Authorisation, and Security....22
19. Form Handling and Validation Strategy.... 23
20. Notification Workflow.... 23
21. User Workflow.... 23
22. Owner Workflow....24
23. User Interface Design Considerations....23
24. Application Screenshots....24-27
25. API Testing with Postman.... 28-28
26. Testing Strategy and Quality Assurance.... 29-30
27. Project Setup and Execution Flow....31-33
28. Challenges Faced and Resolutions....34
29. Project Outcomes.... 35-36
30. Limitations....37
31. Future Enhancements.... 38-39
32. Conclusion.... 40

1. Introduction

The Play Store Application is a full stack web application developed as a capstone submission using the MERN stack (MongoDB, Express.js, React, and Node.js). The platform allows application owners to publish and manage their software, while registered users can browse, search, review, and download applications. The project was built with a strong focus on modular architecture, clean code organisation, and secure authentication using JSON Web Tokens.

The system is built around two clearly defined roles. A User can create an account, sign in, search and browse the application catalogue, filter by category and rating, view detailed application information, write reviews, download applications, and receive notifications when applications they follow are updated. An Owner can create and manage application listings, control which applications are publicly visible, monitor download counts, read user reviews, broadcast update announcements to subscribed users, and manage user accounts through the owner-only User Management page.

The project integrates Formik and Yup for client-side validation, Joi for server-side validation, and covers testing at API integration, component, and end-to-end levels using Jest, Supertest, React Testing Library, and Playwright.

2. Problem Statement

Digital application marketplaces are central to how modern software is distributed and discovered. Developers need a reliable platform to publish and maintain their applications, and users need a straightforward way to find, evaluate, and download the software that suits their needs. Without such a platform, users face difficulties comparing applications side by side, filtering by category or rating, or staying informed about updates.

The Play Store Application directly addresses these gaps. It provides a unified web-based platform with clear role separation, structured application catalogue management, a user review system, category-based discovery, rating-based filtering, and a notification mechanism that keeps both owners and users informed throughout the application lifecycle.

User Stories

1. As a user, I should be able to login, logout, and register into the application.
2. As a user, I should be able to search an application through name and see details like name, description, release date, version, ratings, and genre.
3. As a user, I should be able to see the application under categories like games, beauty, fashion, women, and health.
4. As a user, I should be able to filter applications based on ratings.
5. As a user, I should be able to write reviews and comments for the application.
6. As a user, I should be receiving notifications about updates of the application.

Owner Stories

7. As an owner, I should be able to login and logout into the application. Owner accounts are provisioned through the seeded dataset or by promotion from the User Management page.
8. As an owner, I should be able to perform CRUD operations on my applications.
9. As an owner, I should be able to restrict the visibility of the app from users.
10. As an owner, I should be able to see the count of users who downloaded my application.
11. As an owner, I should be able to see the comments given by users.
12. As an owner, I should be able to receive a notification if my application is downloaded.
13. As an owner, I should be able to announce updates to the users.

3. Need for the Project

This project serves both functional and academic purposes. Functionally, it demonstrates how a marketplace-style platform can be built end-to-end using a modern, JavaScript-based technology stack. Academically, it satisfies the capstone requirement by integrating database modelling, server-side API development, client-side interface design, authentication, validation, and testing into a single cohesive system.

The project illustrates several important engineering principles. It shows how a complete full stack workflow is constructed, starting from database schema design and ending at a deployment-ready frontend build. It demonstrates how role-based access control can be enforced securely using JWT tokens

and protected routes. It maps MongoDB collections and their relationships to a realistic business domain that includes users, applications, reviews, downloads, notifications, and categories.

It delivers a responsive single page application with structured routing and reusable React components. Finally, it serves as a professional portfolio piece that goes beyond a minimal assignment and resembles an actual software product.

4. Project Objectives

The primary objectives of the Play Store Application are listed below.

14. Build a working Play Store-style platform supporting two distinct roles: User and Owner.
1. Implement secure, stateless authentication using JSON Web Tokens.
2. Provide users with full search, category filter, rating filter, review, and download functionality.
3. Provide owners with application CRUD operations, visibility control, download monitoring, and update announcements.
4. Design a MongoDB schema that accurately models the required collections, fields, and relationships.
5. Expose well-structured RESTful APIs through Express.js using a modular, controller-service architecture.
6. Build a responsive React single page application with clearly defined public and protected routes.
7. Apply client-side validation with Formik and Yup, and server-side validation with Joi.

8. Achieve test coverage at the API integration, component, service, and end-to-end levels.
9. Produce documentation that supports project setup, manual testing, viva preparation, and final report submission.

5. Scope of the Project

The project scope covers the complete lifecycle of both user and owner interactions within the platform. On the user side, this includes account registration, login, logout, browsing the application catalogue, searching by name, filtering by category and minimum rating, viewing full application details, writing reviews, and downloading applications. On the owner side, this includes creating, editing, and deleting application records, toggling visibility, monitoring download statistics, reading user reviews, publishing update announcements, and managing user accounts through the owner-only User Management page.

The project also includes a database seeding script that pre-populates the system with realistic demonstration data, and a set of operational documents covering MongoDB setup, API verification with Postman, and GitHub workflow guidance.

Features that fall outside the current scope include payment processing, cloud-based file uploads, real-time push or email notifications, multi-tenant deployment, social login, and native mobile application packaging. These are all valid future enhancements but are not required by the capstone brief.

6. Stakeholders and User Roles

Four groups of stakeholders are relevant to this project: end users, application owners, evaluators, and the developer.

User

A User is anyone who visits the platform to discover and download software. Users can register, sign in, search and filter the application catalogue, view detailed application pages, post reviews and ratings, download applications, and receive in-app notifications when an application they downloaded releases an update.

Owner

An Owner is a developer or publisher who wants to distribute software through the platform. Owners can sign in using a seeded owner account or an account promoted by an existing owner, create and manage application records, update application metadata, toggle the visibility of their applications, monitor download counts and user reviews through a dedicated dashboard, announce updates to users who have downloaded their applications, and manage user accounts from the owner-only User Management page.

Evaluator

An evaluator reviews the project to determine whether it satisfies the capstone requirements, follows sound architectural principles, and has been properly documented and tested. The seed data and operational guides have been prepared with the evaluator's ease of use in mind.

Developer

The developer is responsible for all phases of the project: design, implementation, integration, testing, documentation, and maintenance.

7. Requirement Analysis

The capstone problem statement defines a set of functional requirements for each role. On the user side, the system must support registration, login, and logout; searching applications by name and viewing details such as name, description, release date, version, ratings, and genre; browsing by categories including games, beauty, fashion, women, and health; filtering by rating; writing reviews and comments; and receiving notifications about application updates.

On the owner side, the system must support registration, login, and logout; full CRUD operations on their applications; visibility control to restrict or restore public access to an application; viewing the number of downloads; reading user comments; receiving a notification when their application is downloaded; and sending update announcements to users.

The implemented system satisfies all these requirements through six modules: the authentication module, the application catalogue module, the review module, the download module, the notification module, and the owner dashboard module. An additional owner-only User Management module enables promoting or deleting normal user accounts without altering the two-role architecture.

8. Development Methodology and Sprint Execution

The project followed a staged delivery approach structured across three sprints, aligned to the capstone sprint plan.

Sprint I — Foundation

- MongoDB collections designed with their relationships (User, Application, Category, Review, Download, Notification)
- Base authentication routes for registration, login, and logout built with Express.js and JWT
- React project skeleton created with initial page components and React Router DOM configured

Sprint II — Core Functionality

- Application search and filtering by name, category, and minimum rating implemented
- Owner-side application CRUD, visibility toggle, and feature editing added
- User review module built with rating and comment submission
- JWT-based authentication and role-based route protection applied across frontend and backend
- Component-level and end-to-end testing frameworks set up

Sprint III — Integration and Polish

- Service layer formalised to separate business logic from controllers
- Controllers created to handle RESTful API requests and delegate to service functions
- Notification logic implemented for both owner download alerts and user update notifications
- Frontend and backend fully integrated, tested, and UI refined for responsiveness.

9. Technology Stack and Justification

Frontend

The frontend is built with React and React Router DOM. React was chosen for its component-based model, efficient state management, and strong ecosystem. Material UI provides the component library, offering a consistent design system, responsive grid layout, and pre-built UI primitives. Axios handles all HTTP communication with the backend. Formik and Yup together manage form state and client-side validation.

Technology	Purpose
React.js	Component-based SPA framework
React Router DOM	Client-side routing and protected routes
Material UI	Responsive component library and theme system
Axios	HTTP client with JWT Bearer token interceptor
Formik + Yup	Form state management and client-side validation

Backend

The backend is built with Node.js and Express.js. Express was selected for its lightweight routing model, flexible middleware system, and clear structure for REST API design. Mongoose is used to define and validate MongoDB documents. JSON Web Tokens handle stateless authentication. bcryptjs secures stored passwords. Joi schemas validate all incoming request payloads. Helmet adds recommended security headers, and express-mongo-sanitize prevents MongoDB operator injection attacks.

Technology	Purpose
Node.js + Express.js	REST API server and routing framework
Mongoose	ODM for MongoDB schema definition and querying

jsonwebtoken	JWT signing and verification for auth
bcryptjs	Password hashing before database storage
Joi	Server-side request body validation
Helmet	HTTP security headers middleware
express-mongo-sanitize	Prevents MongoDB operator injection attacks
Jest + Supertest	Backend API integration testing
Playwright	End-to-end browser smoke testing

Database

MongoDB was selected because the application's data model is catalogue-driven and document-oriented. Different application records can have varying fields without requiring schema migrations. Mongoose adds a structured schema layer on top of MongoDB, providing validation, type casting, and relationship references through ObjectId fields.

10. System Architecture

The system follows a standard client-server architecture. The React frontend runs in the user's browser and communicates with the Express backend through HTTP requests over a RESTful API. The backend processes those requests, applies business logic through service modules, and reads from or writes to MongoDB through Mongoose models. The two tiers are fully separated: the frontend does not directly access the database, and the backend does not render any HTML.

Authentication is the entry point for protected operations. When a user registers or logs in, the backend returns a signed JWT token. The frontend stores this token in local storage and attaches it to every subsequent API request through an Axios interceptor using the standard Bearer token format. On the backend, a middleware function verifies the token on each protected route and extracts the user's identity and role. A second middleware then enforces role-based access, ensuring that owner-only routes reject requests from regular users.

Three-Tier Architecture Overview

- Presentation Layer: React.js SPA runs in the browser, handles all UI rendering and routing via React Router DOM.
- Application Layer: Node.js and Express.js process API requests through a controller-service pattern, with Joi validation and JWT middleware at every entry point.
- Data Layer: MongoDB stores all collections; Mongoose provides schema validation, type casting, and ObjectId population for related documents.

Frontend-to-Backend Data Flow

```
User Action
   ↓
React Component (Formik form / button click)
   ↓
Axios API Call (with JWT Bearer token header)
   ↓
Express Route → Joi Validation Middleware → Auth Middleware
   ↓
Controller → Service Function
   ↓
Mongoose Model → MongoDB
```

11. Entity Relationship Diagram

The Entity Relationship Diagram (ERD) represents the six MongoDB collections of the Play Store system and their relationships. The diagram follows Crow's Foot notation with one-to-many cardinality on all relationships.

□ Entity Relationship Diagram — Play Store System

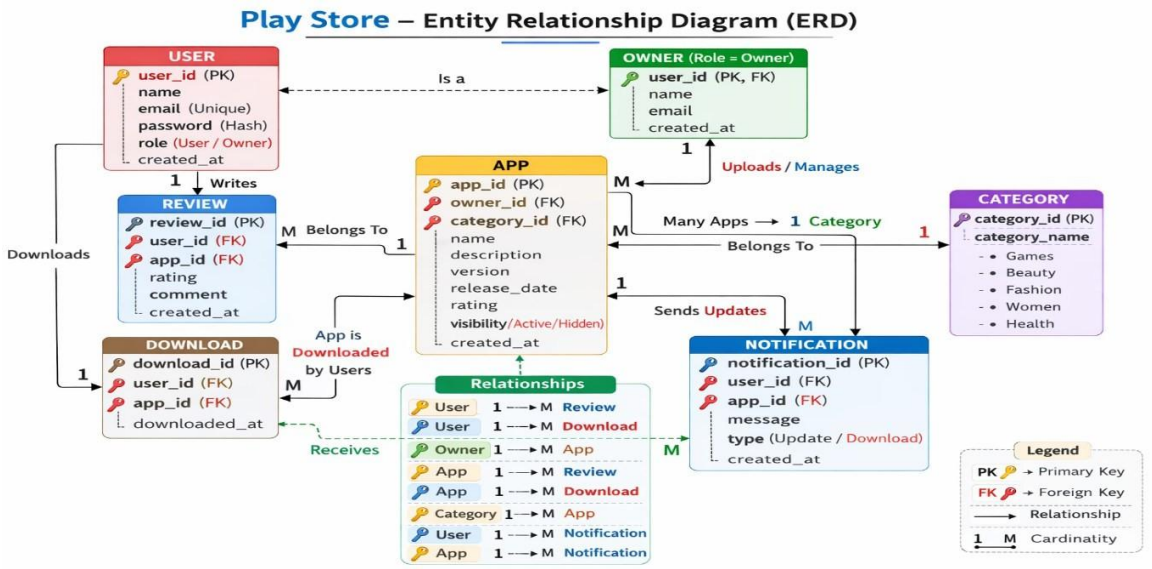


Figure: Entity Relationship Diagram — Play Store System

Entities and Key Attributes

Entity	Key Fields
User	_id, name, email, password (hashed), role (user|owner), subscribedApps[]
Application	_id, ownerId (FK), categoryId (FK), name, version, genre, visibility, rating, averageRating, reviewCount, downloadCount, features[]
Category	_id, name (enum: Games/Health/Fashion/Beauty/Women)
Review	_id, userId (FK), appId (FK), rating (1-5), comment, createdAt
Download	_id, userId (FK), appId (FK), downloadedAt
Notification	_id, recipientId (FK), appId (FK), type (app_updated|app_downloaded), message, isRead

Relationships

- User (Owner) 1 ——— * Application: one owner publishes many applications
- Category 1 ——— * Application: one category contains many applications
- User 1 ——— * Review: one user writes many reviews
- Application 1 ——— * Review: one application has many reviews
- User 1 ——— * Download: one user downloads many applications
- Application 1 ——— * Download: one application has many download records
- Application 1 ——— * Notification: one application triggers many notifications
- User 1 ——— * Notification: one user receives many notifications

Note: The Application document stores downloadCount, reviewCount, rating, and averageRating directly. These are updated when reviews and downloads are recorded. The ERD reflects the actual schema fields in the Application Mongoose model.

12. Project Folder Structure

The project follows a monorepo structure with backend and frontend as sibling directories, managed via npm workspaces from the root package.json.

Backend Structure

```
backend/
├── config/
│   ├── db.js          ← MongoDB connection
│   └── env.js
├── controllers/    ← Thin request handlers
│   ├── appController.js
│   ├── authController.js
│   ├── downloadController.js
│   ├── notificationController.js
│   ├── reviewController.js
│   └── userController.js
├── middlewares/
│   ├── authMiddleware.js   ← JWT verify
│   ├── authMiddleware.js   ← JWT verify + role check
│   ├── errorMiddleware.js
│   └── validate.js         ← Joi validation runner
├── models/         ← Mongoose schemas
│   ├── Application.js
│   ├── Category.js
│   ├── Download.js
│   ├── Notification.js
│   ├── Review.js
│   └── User.js
├── routes/
├── services/       ← Business logic layer
├── validators/     ← Joi schemas
├── utils/
├── tests/
└── server.js
```

Frontend Structure

```
frontend/src/
├── components/
│   ├── apps/          ← AppCard, AppFilters, AppForm
│   ├── common/        ← EmptyState, LoadingScreen, PageHero
│   ├── layout/        ← AppShell (navigation wrapper)
│   ├── notifications/ ← NotificationList
│   └── reviews/       ← ReviewForm, ReviewList
├── contexts/
│   └── AuthContext.jsx  ← Global auth state
├── hooks/
│   └── useAuth.js
├── pages/
│   ├── Home.jsx | Login.jsx | Register.jsx
│   ├── AppListing.jsx | AppDetails.jsx
│   ├── OwnerDashboard.jsx | AddApp.jsx | EditApp.jsx
│   ├── Notifications.jsx | UserProfile.jsx
│   └── Notifications.jsx | UserProfile.jsx
├── routes/
│   ├── AppRoutes.jsx
│   └── ProtectedRoute.jsx  ← Auth/role guard
├── services/       ← Axios modules per resource
├── styles/         ← MUI theme + global CSS
└── utils/          ← JWT storage, date formatters
```

13. Frontend Design and Module Structure

The frontend source code is organised inside the frontend/src directory according to a responsibility-based folder structure. The components folder holds reusable building blocks: application cards, the navigation bar, category filter panels, review lists, and page hero sections. These components are shared across multiple pages.

The pages folder contains one file per route, covering the Home page, Application Listing, Application Details, Login, Register, User Profile, Notifications, Owner Dashboard, Add Application, and Edit Application. Each page composes components and interacts with the services layer.

The services folder contains Axios-based modules, one per resource type: authentication, applications, categories, reviews, downloads, and notifications. Each module exports functions that map to specific API endpoints, keeping API logic centralised and easy to update. The contexts

folder holds the authentication context, which stores the current user's session state and exposes it through a custom useAuth hook.

The routes folder exports a ProtectedRoute wrapper that redirects unauthenticated users to the login page and prevents regular users from accessing owner-only pages. Navigation between pages happens entirely through React Router without full page reloads, making the application a true single page application.

14. Backend Design and Module Structure

The backend source code follows a layered architecture that separates routing, request handling, business logic, and data access into distinct folders. The config folder holds database connection logic and environment variable loading. The models folder contains Mongoose schema definitions for every collection. The routes folder maps HTTP methods and URL paths to controller functions.

The controllers folder contains thin handler functions that receive a request, call the appropriate service function, and return the response. The services folder contains the actual business logic: deciding what data to fetch, how to compute derived values, when to create notifications, and how to handle edge cases. This keeps controllers simple and makes services independently testable.

The middlewares folder contains the JWT authentication check, the role-based authorisation check, a global error handler, and request-level validation middleware that runs Joi schemas before the controller is reached. The validators folder stores the Joi schemas themselves. The utils folder holds JWT signing and verification helpers, along with shared constants such as role names and notification types.

15. Database Design and Relationship Model

The database is organised into six collections, each representing a distinct entity in the domain model. The Users collection stores account credentials, the user's role, and a list of application IDs the user has subscribed to through downloads. The Categories collection stores the category names and URL slugs used to group and filter applications.

The Applications collection is the central collection, storing all metadata about a published application along with a reference to its owner and its category. The Reviews collection stores individual ratings and comments submitted by users, each linked to both a user and an application. The Downloads collection records each download event, linking a user to an application with a timestamp. The Notifications collection stores in-app messages for both users and owners, with a type field that distinguishes between download notifications and update notifications.

The relationships between collections follow a clear pattern. One owner user can create many applications. One application can accumulate many reviews and many download records. Each notification is addressed to one recipient and relates to one application. Users maintain a subscribedApps array that is updated whenever a download is recorded, providing the list of recipients when an owner announces an update.

16. Detailed Collection Design

The Users collection stores the fields name, email, password (saved as a bcrypt hash), role (either 'user' or 'owner'), and subscribedApps (an array of ObjectId references to Application documents).

The Categories collection stores name, slug, a short description, and an isActive flag that allows categories to be disabled without deletion.

The Applications collection stores name, description, version string, releaseDate, a reference to its Category, genre, a computed averageRating, a reviewCount, a visibility field, a reference to its owner in ownerId, an iconUrl string, a featureHighlights array, a downloadCount integer, and a lastAnnouncement field.

The Reviews collection stores userId, appId, a rating integer from one to five, a comment string, and a createdAt timestamp. The Downloads collection stores userId, appId, and downloadDate. The Notifications collection stores recipientId, recipientRole, a type field (‘app_updated’ or ‘app_downloaded’), a title, a message, a relatedAppId reference, an isRead boolean, and a createdAt timestamp.

17. API Design and Endpoint Summary

All API routes are prefixed with /api. The table below summarises all available endpoints.

Endpoint	Description
POST /api/auth/register	Create a new user account
POST /api/auth/login	Authenticate and return a JWT token
POST /api/auth/logout	Complete the session on the client side
GET /api/auth/me	Return the profile of the authenticated user
GET /api/apps	List all apps with optional search/filter params
GET /api/apps/search	Name-based keyword search
GET /api/apps/:id	Retrieve a single application’s full detail
POST /api/apps	Create a new application (Owner only)

PUT /api/apps/:id	Update application metadata (Owner only)
PATCH /api/apps/:id/visibility	Toggle public visibility flag (Owner only)
DELETE /api/apps/:id	Delete an application (Owner only)
POST /api/apps/:id/announce-update	Notify all subscribed users (Owner only)
GET /api/apps/owner/dashboard/summary	Aggregated owner dashboard statistics
POST /api/reviews	Submit a new review
GET /api/reviews/app/:id	Retrieve all reviews for an application
DELETE /api/reviews/:id	Remove a review
POST /api/downloads	Record a download event
GET /api/downloads/owner	Download records for owner's apps
GET /api/notifications	Retrieve notifications for current user
PATCH /api/notifications/:id/read	Mark a single notification as read
PATCH /api/notifications/read-all	Mark all notifications as read
GET /api/categories	List all active categories
GET /api/users	List all user accounts (Owner only)
PUT /api/users/:id/role	Promote a user to owner role (Owner only)
DELETE /api/users/:id	Delete a normal user account (Owner only)

18. Authentication, Authorisation, and Security

Authentication is handled using JSON Web Tokens. When a user successfully registers or logs in, the backend signs a token containing the user's ID and role, and returns it to the frontend. The frontend stores this token in local storage and attaches it to all subsequent API requests via an Axios interceptor in the standard Authorization: Bearer <token> format.

Authorisation is role-based and enforced by two middleware functions. The first middleware verifies the token and attaches the decoded user payload to the request object — if the token is missing, expired, or invalid, the request is rejected with a 401 status. The second middleware checks the role field in the decoded payload for owner-only routes — if the role is not 'owner', the request is rejected with a 403 status.

Additional security measures include: bcryptjs hashing passwords before storage; Joi schemas validating all request bodies before reaching controller logic; Helmet setting appropriate HTTP security headers; express-mongo-sanitize stripping MongoDB operator characters from inputs; and centralised error handling ensuring stack traces are never exposed to clients.

19. Form Handling and Validation Strategy

Client-side form management is handled with Formik for state and Yup for schema-based validation rules. This combination is applied to all major forms: user registration, login, application creation, application editing, and review submission. Formik tracks field values, touched states, and error messages, while Yup defines the validation rules declaratively. Errors are shown inline beneath each field as the user types or moves away from a field.

Server-side validation uses Joi schemas defined in the validators folder. A validation middleware function runs the appropriate schema against the request body before passing control to the controller. If validation fails, the middleware returns a 400 response with a descriptive error message. This dual-layer approach means that even if a request bypasses the frontend entirely (e.g. via Postman), the backend will still reject any data that does not meet the required format or constraints.

20. Notification Workflow

The notification system supports two distinct event-driven workflows that together satisfy the requirements for both user and owner notification.

Workflow 1: Download Notification to Owner

This workflow is triggered when a user downloads an application. The download service creates a new record in the Downloads collection and simultaneously creates a notification document in the Notifications collection addressed to the application's owner. This notification has the type 'app_downloaded' and informs the owner which user downloaded which application and when.

Workflow 2: Update Announcement to Users

This workflow is triggered when an owner announces an update through the POST /api/apps/:id/announce-update endpoint. The notification service queries the Users collection for all users whose subscribedApps array contains the relevant application ID. It then creates one notification document per subscribed user, each with the type 'app_updated'. Subscription is established automatically whenever a user downloads an application for the first time.

21. User Workflow

A typical user journey through the platform proceeds as follows. The user arrives at the home page and either registers for a new account or logs in with existing credentials. Once authenticated, the frontend stores the JWT token and automatically restores the session on any subsequent page reload.

From the home page, the user can see featured applications. On the Applications page, the user can type a search term, select a category from the filter panel, or set a minimum rating to narrow results in real time. Clicking on any application card opens the Application Details page, which shows the

full description, release date, version number, genre, owner name, total download count, average rating, and all user reviews.

From the details page, the user can click Download to record the download and add the application to their subscribed list. They can also submit a review with a star rating and a written comment. If they return to the Notifications page later and the owner has announced an update, an unread notification will be waiting for them.

22. Owner Workflow

An owner's workflow begins at login using a seeded owner account or a promoted account, after which they are directed to the Owner Dashboard. The dashboard presents a summary of all their published applications, showing the download count and most recent user reviews for each one at a glance.

To publish a new application, the owner navigates to the Add Application page and fills in a form with the application's name, description, version string, category, genre, release date, and an icon URL. The application is saved to the database and immediately appears in the owner's dashboard. The owner can edit any of this metadata at any time from the dashboard using the Edit Application page.

The owner can toggle the visibility of any application. When an application is hidden, it disappears from all public-facing listings while remaining accessible in the owner dashboard. When a new version is ready, the owner submits an update announcement, which the system delivers as notifications to all subscribed users. Separately, the owner is notified each time any user downloads one of their applications. The owner can also open the User Management page to view registered accounts, promote a normal user to

owner, or delete a normal user account. Deleting a user removes their reviews, downloads, and personal notifications and recalculates affected application counts.

23. User Interface Design Considerations

The frontend interface was designed to feel balanced, professional, and easy to navigate without requiring any prior familiarity with the system. The overall visual theme uses Material UI's component library with a custom colour scheme, consistent spacing, and clear typographic hierarchy.

Application cards on the Home page and Applications listing page are standardised to equal height and width, with fixed-size image containers that maintain proportions regardless of icon dimensions. The navigation bar distinguishes between the public-facing navigation visible to all users and the owner-specific navigation that includes the Dashboard link.

All layouts have been tested and adjusted for desktop, tablet, and mobile screen sizes using Material UI's responsive grid and breakpoint system, ensuring that the interface remains readable and usable across devices. A shared footer with the project support contact email appears consistently across all pages to provide a clean and professional presentation.

24. Application Screenshots

The following screenshots illustrate the key pages of the Play Store application. Insert your screenshots in the placeholders below.

☑RegistrationPage

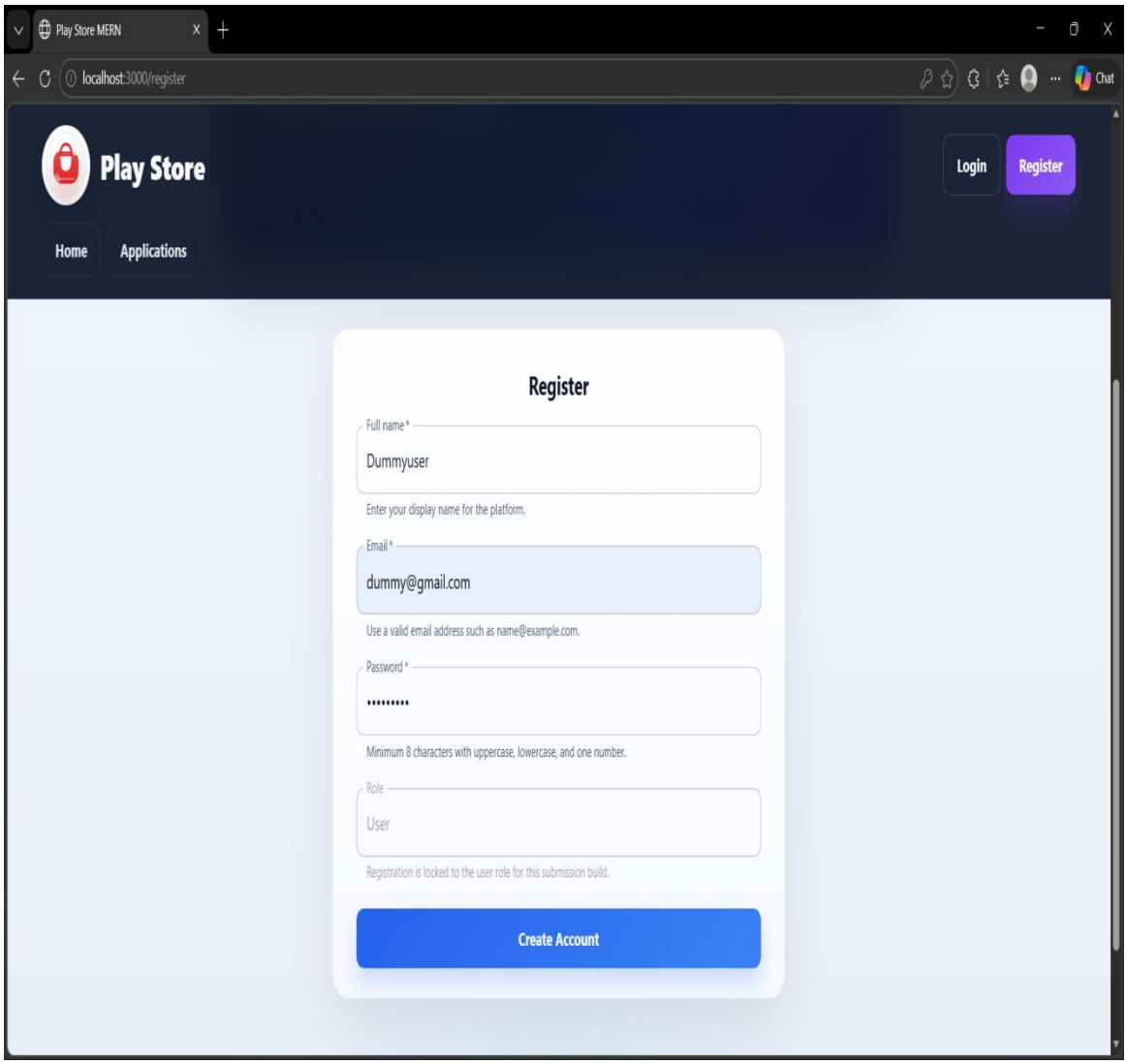


Figure: Registration Page

☑ Login Page

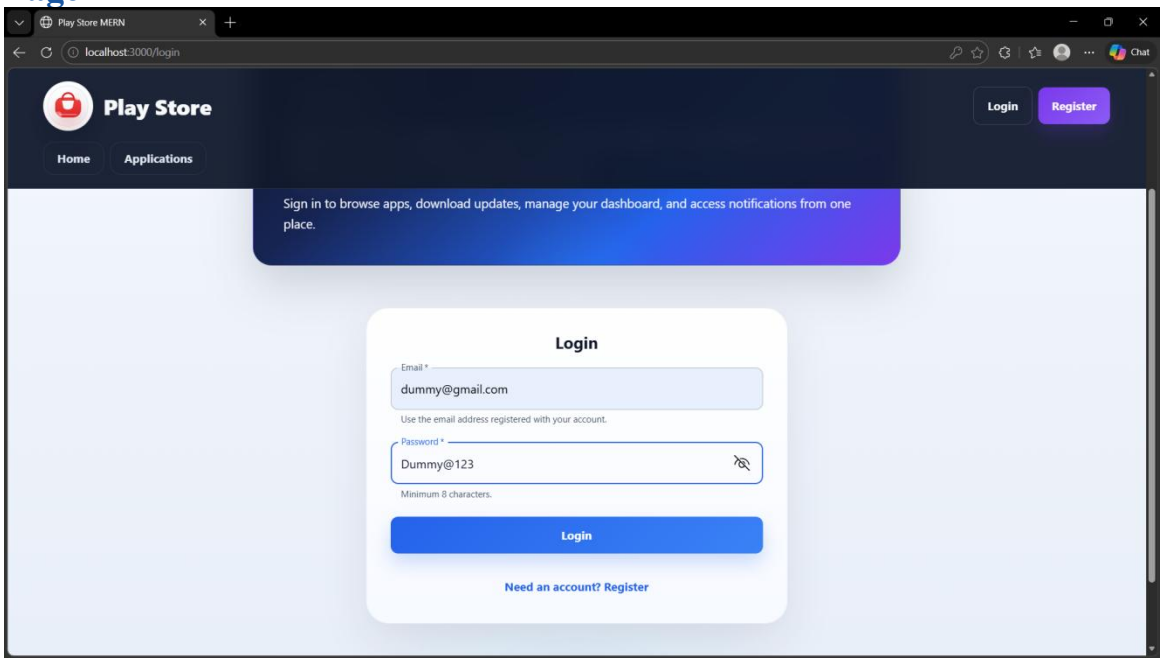


Figure: Login Page

☑ Home Page — Application Catalogue

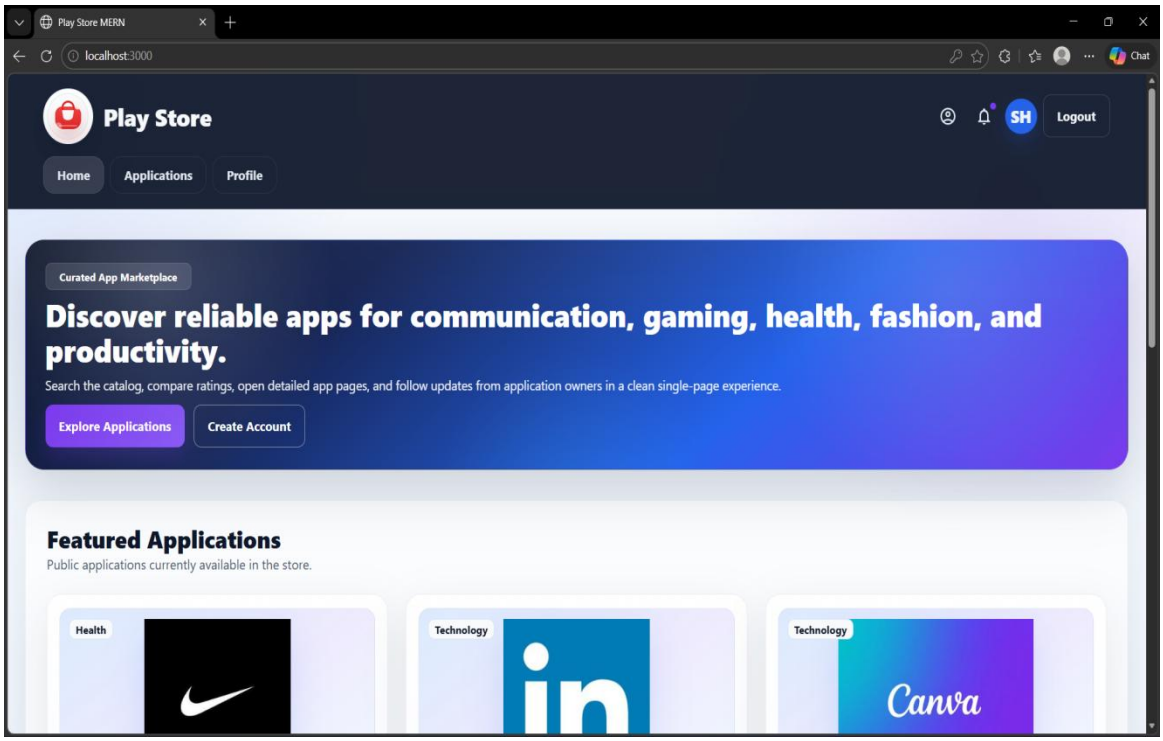


Figure: Home Page — Application Catalogue

☑ User Dashboard — Notifications Page

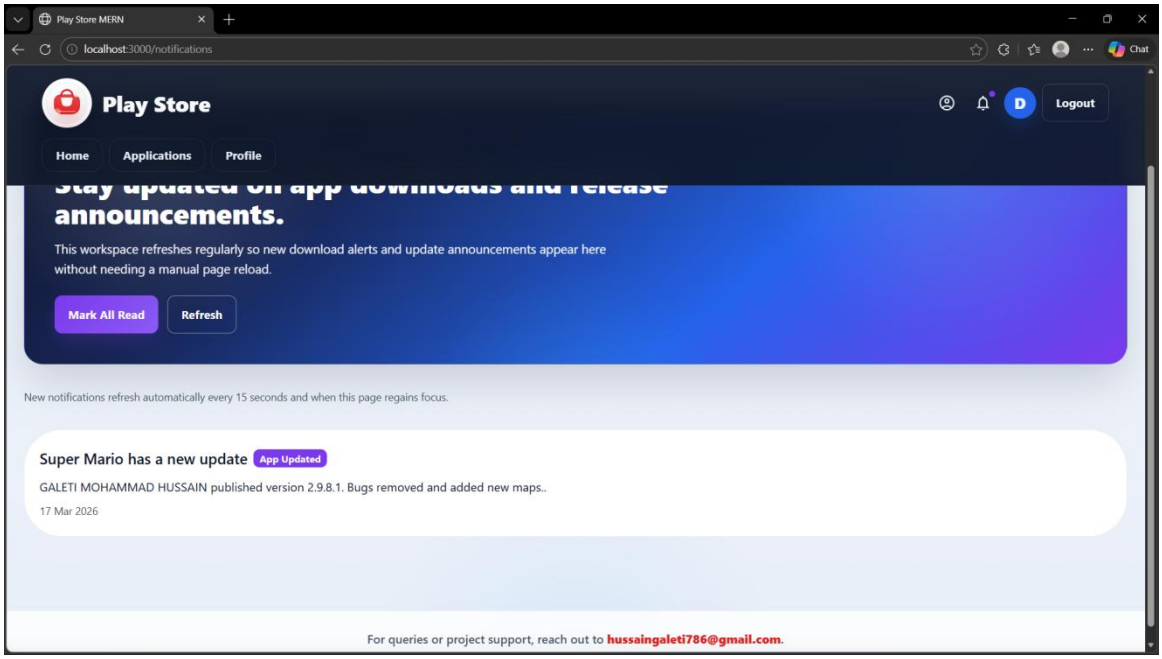


Figure: User Dashboard — Notifications Page

☑ Owner Dashboard — Application Management

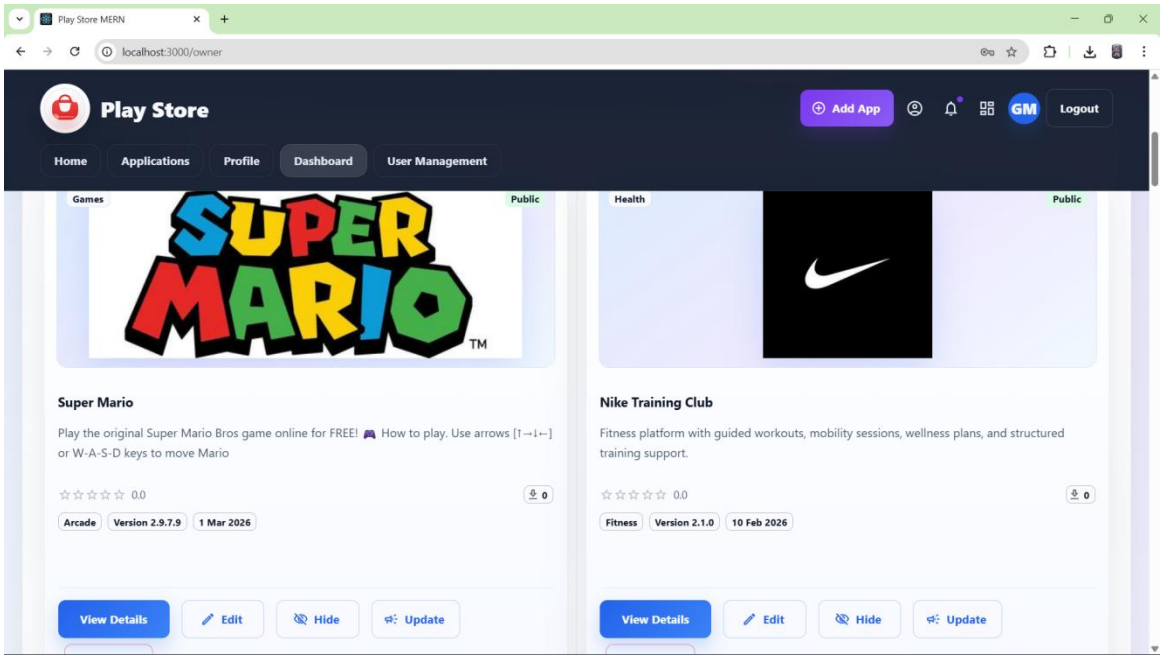


Figure: Owner Dashboard — Application Management

☑ Add Application Page (Owner)

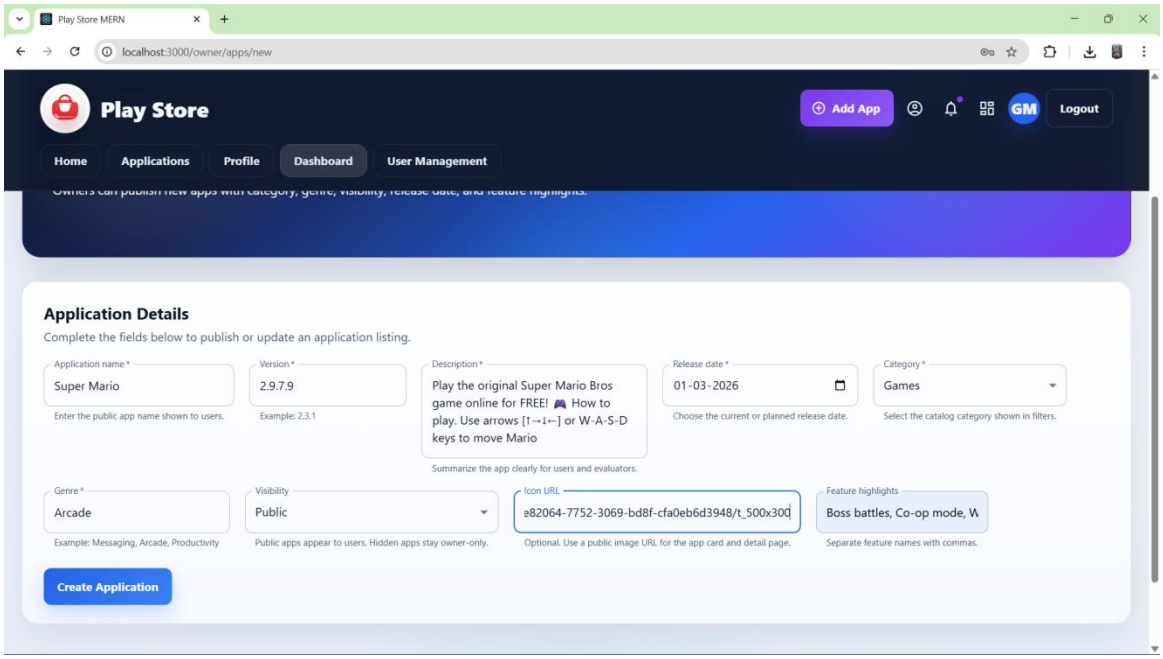
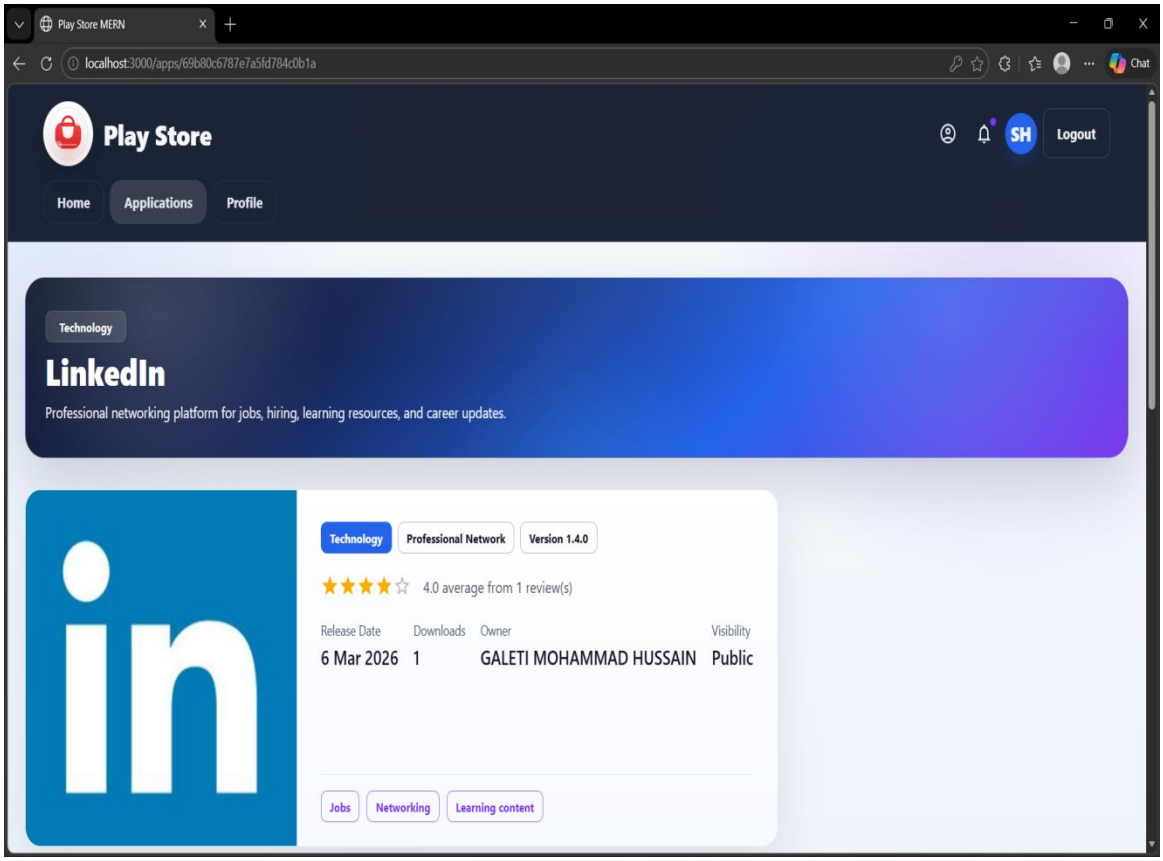


Figure: Add Application Page (Owner)

☑ Application Details Page



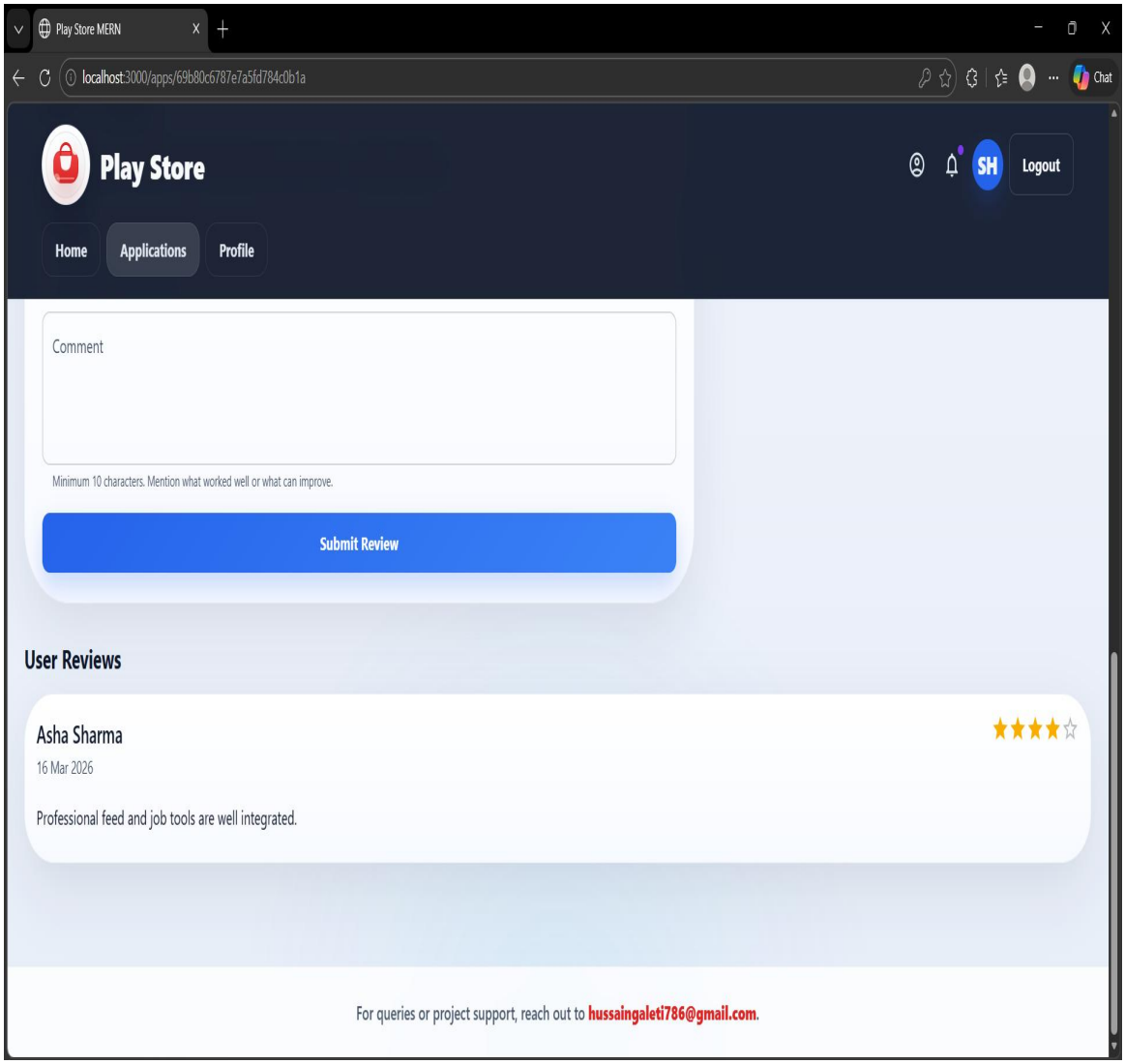


Figure: Application Details Page

25. API Testing with Postman

All API endpoints were manually verified using Postman. The following screenshots show the key API operations tested during development. Insert your Postman screenshots in the placeholders below.

☑ POST /api/auth/register

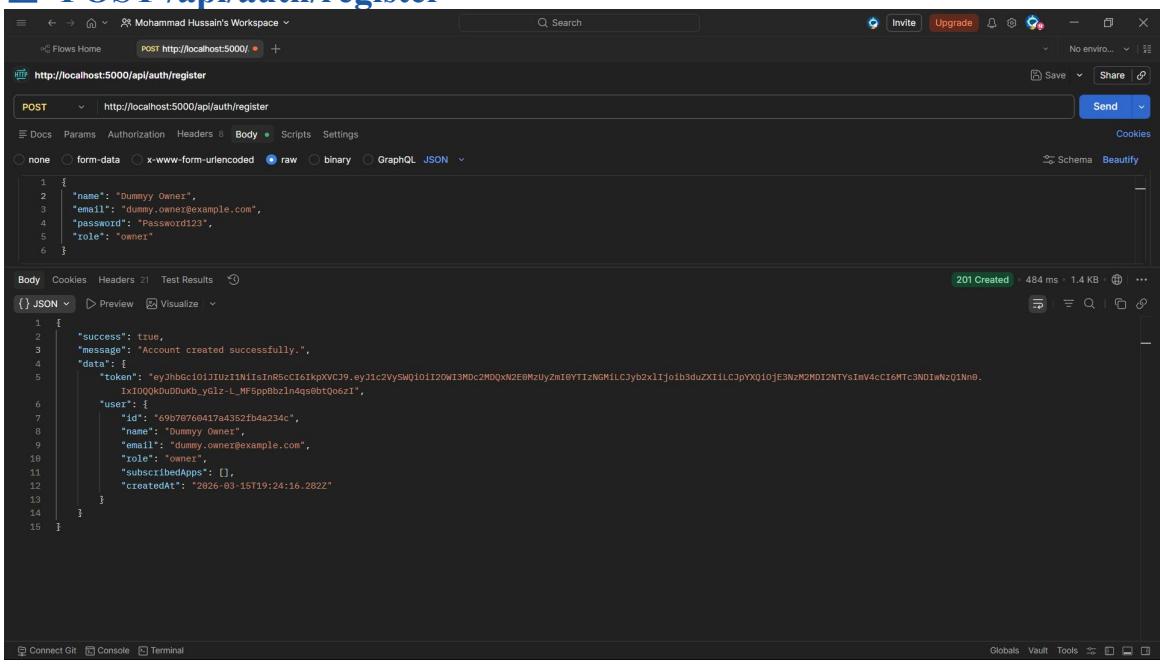


Figure: POST /api/auth/register — User Registration

☑ POST /api/auth/login — JWT Token Response

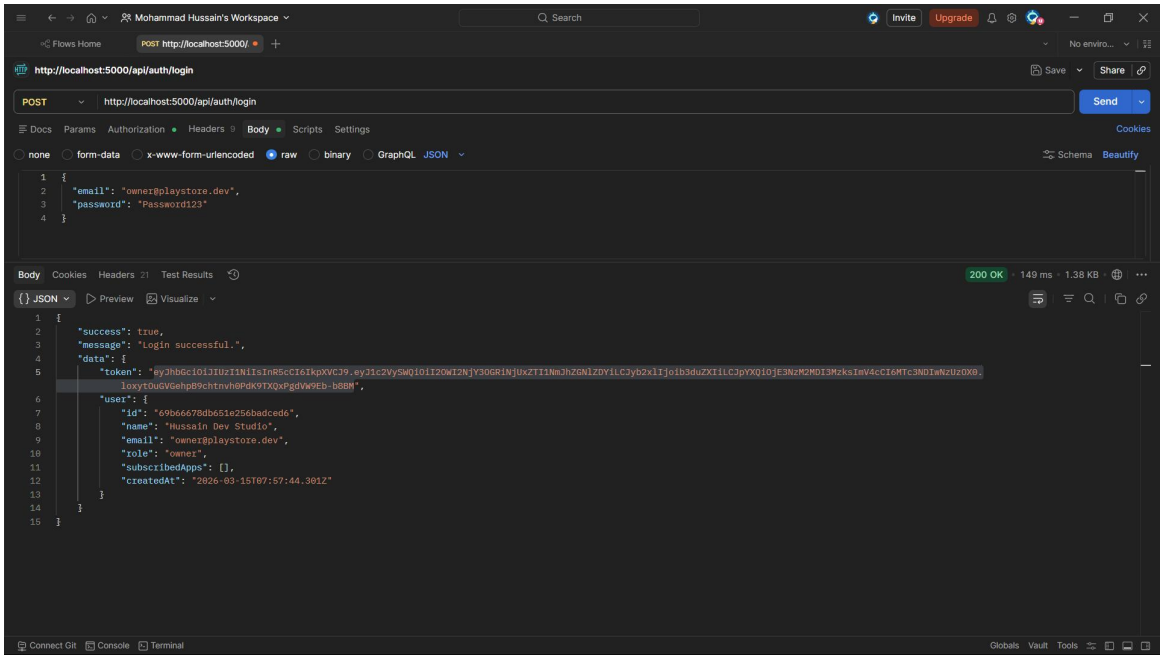


Figure: POST /api/auth/login — JWT Token Response

☑ GET /api/apps — Application Catalogue with Filters

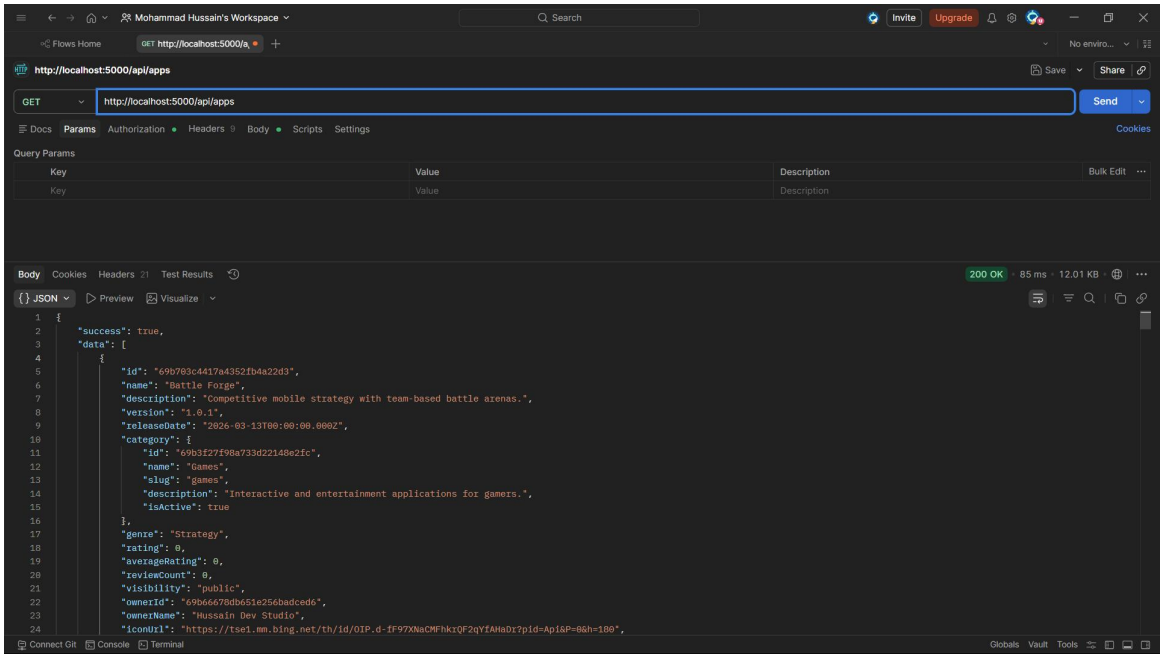


Figure: GET /api/apps — Application Catalogue with Filters

☑ POST /api/apps — Add Application (Owner, with Auth Header)

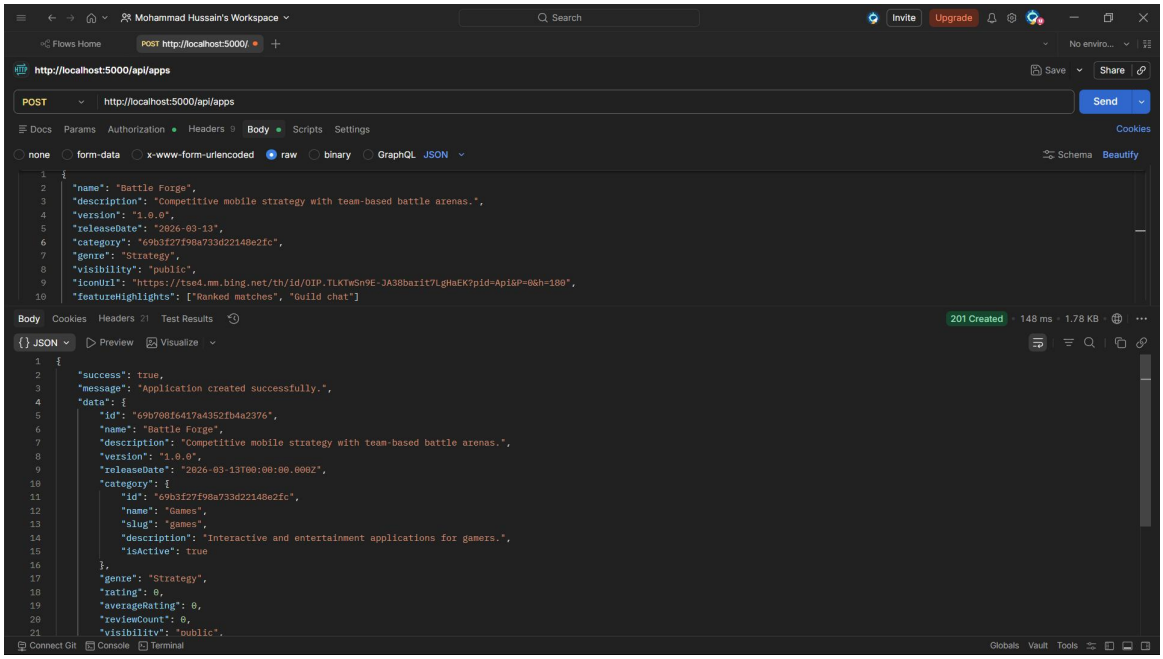


Figure: POST /api/apps — Add Application (Owner, with Auth Header)

☑ DELETE /api/apps/:id — Delete Application (Owner)

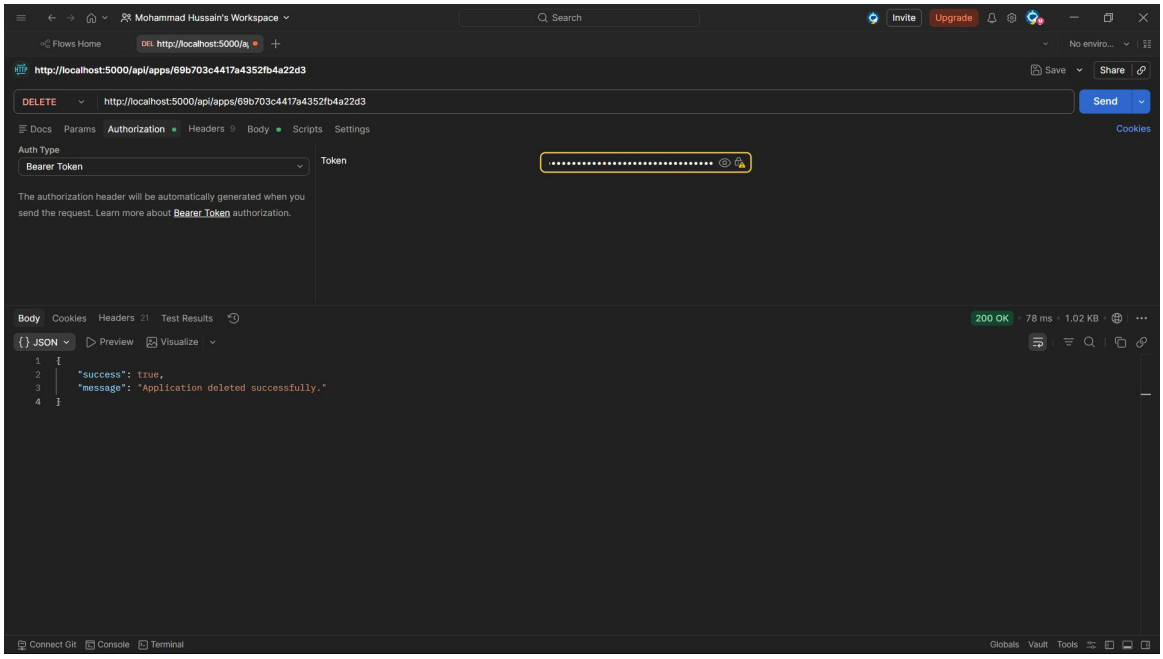


Figure: DELETE /api/apps/:id — Delete Application (Owner)

26. Testing Strategy and Quality Assurance

Testing was carried out at four levels to provide confidence across the entire stack.

API Integration Testing (Jest + Supertest)

Jest and Supertest run integration tests against the live Express application. These tests cover the authentication flow, application CRUD operations with role-based access control, review creation and retrieval, download recording, and notification generation. The test environment uses a local MongoDB fallback directly on Windows for stable execution. On other environments, mongodb-memory-server may be used when available. This ensures tests run reliably across different operating systems without leaving persistent state between runs.

Component and Service Testing (React Testing Library + Jest)

React Testing Library and Jest verify that individual components render correctly with given props, that form validation messages appear as expected, and that service modules make the correct Axios calls with the right parameters.

End-to-End Testing (Playwright)

Playwright runs browser-based smoke tests that launch the full application and verify that the home page loads, navigation links work, and the registration page is reachable and interactive. Build verification was also performed by running the react-scripts production build to confirm the frontend compiles cleanly.

Dependency Summary

Layer	Testing Tool
Backend API	Jest + Supertest + local MongoDB fallback (Windows) / mongodb-memory-server
Frontend Components	React Testing Library + Jest
End-to-End	Playwright (Chrome / Edge)
Manual API	Postman

27. Project Setup and Execution Flow

Setting up and running the project follows a straightforward sequence. Because the project uses npm workspaces, all dependencies for both the backend and frontend are installed with a single npm install command run from the root directory.

```
# Step 1 — Install all dependencies from root
```

```
npm install

# Step 2 — Configure environment files
# Copy backend/.env.example → backend/.env  (set MONGO_URI,
JWT_SECRET)
# Copy frontend/.env.example → frontend/.env  (set
REACT_APP_API_URL)

# Step 3 — Seed the database with demo data
npm run seed

# Step 4 — Start both servers concurrently from root
npm run dev
# Backend runs on http://localhost:5000
# Frontend runs on http://localhost:3000
```

The seed script restores a snapshot of the current working dataset into MongoDB. The present snapshot contains 7 categories, 5 users, 16 applications, 16 reviews, 17 downloads, and 13 notifications. All 16 applications in the snapshot are public. The root owner login is hussaingaleti786@gmail.com with password Hussain@786. Opening a browser at localhost:3000 presents the application, and the seeded demo credentials can be used to verify both the User and Owner roles without any manual data entry.

28. Challenges Faced and Resolutions

Several practical challenges arose during the development process and required deliberate resolution.

Challenge 1: Schema Document Discrepancy

The main capstone problem statement and a secondary schema document used slightly different terminology and collection names. This was resolved by treating the main problem statement as the primary source of truth and using the secondary document only as a reference for field-level detail.

Challenge 2: Empty Query Parameter Errors

Runtime validation errors occurred when empty query parameters were passed to the backend when the user had not selected a filter value. The fix involved sanitising all filter parameters on the frontend before building the query string, and adding fallback handling on the backend to ignore empty or undefined filter values.

Challenge 3: Visual Consistency Across Components

Cards added at different stages of development had slightly different padding, font sizes, and button placements. This was resolved during Sprint Three through a systematic layout review that applied Material UI theme tokens uniformly across all card and grid components.

Challenge 4: Documentation Drift

As the codebase evolved, earlier drafts of the README and API guides described features or file paths that no longer matched the current implementation. This was resolved by treating documentation updates as a mandatory part of each sprint's completion criteria and doing a final full review before submission.

29. Project Outcomes

The completed project delivers the following concrete outcomes.

- A fully working MERN Play Store system with role-based functionality for both User and Owner roles, built and tested.
- A modular Express backend with a clear service layer, Joi validation at every entry point, and JWT security.
- A React single page application with responsive layouts for both public and owner-facing pages.

- A MongoDB schema covering six collections with clearly defined relationships supporting the full range of required operations.
- A snapshot-based seed script that restores the current working dataset (7 categories, 5 users, 16 applications, 16 reviews, 17 downloads, 13 notifications) immediately after setup.
- A documentation suite covering project setup, API verification, JWT flow, GitHub instructions, and this capstone report.

30. Limitations

The project is scoped to the capstone requirements and therefore has some limitations worth noting.

- Application icon images are provided as URL strings rather than actual file uploads, since cloud storage integration was outside scope.
- Notifications are stored in MongoDB and displayed in the frontend, rather than delivered through external channels such as email or mobile push services.
- Deployment automation through CI/CD pipelines or cloud hosting configuration has not been included.
- Application search is limited to name-based matching and does not support full-text search across descriptions or features.
- The system does not include advanced analytics, trending algorithms, or personalised recommendation features.

None of these limitations affect the functionality required by the capstone brief, but they represent clear starting points for extending the system in the future.

31. Future Enhancements

Several meaningful enhancements could extend this project beyond its current capstone scope.

Enhancement	Description
Real-time Notifications	Replace database polling with WebSockets or Socket.io for instant alerts
Cloud File Storage	AWS S3 or Cloudinary for actual APK/icon uploads instead of URL strings
Advanced Search	Full-text search across names, descriptions, and feature highlights
Recommendation Engine	ML-based suggestions using download history and ratings
Admin Management Panel	Platform-level moderation: approve apps, suspend accounts, remove content
Analytics Dashboard	Time-series download charts, rating trends, and user engagement metrics
Mobile Application	React Native or Flutter wrapper consuming the existing REST API
Payment Integration	Stripe or Razorpay for paid apps and in-app purchases
Enhanced Security	Rate limiting, OAuth (Google login), and two-factor authentication (2FA)
Performance Optimisation	Redis caching, MongoDB indexing, and CI/CD pipeline configuration

32. Conclusion

The Play Store Application demonstrates a complete, well-structured implementation of the capstone problem statement using the MERN stack. The system supports two user roles with clearly separated functionality, secure JWT-based authentication, a full application catalogue with search and filter capabilities, user reviews, download tracking, owner-side administration through a dedicated dashboard, a two-way notification workflow, a responsive interface across device sizes, and test coverage at multiple levels of the stack.

The project was built with sustained attention to architecture, maintainability, and presentation quality. Each layer of the system, from the MongoDB schema to the React component tree, was designed to be readable, extendable, and easy to navigate during a viva or code review. Security is layered throughout: bcryptjs hashes passwords, JWT tokens are verified on every protected request, Helmet adds HTTP security headers, and express-mongo-sanitize prevents injection attacks.

The seed data, documentation, and operational guides ensure that any evaluator can run the full system within minutes of cloning the repository. As a result, the project serves not only as a capstone submission but also as a practical demonstration of full stack development skill. It is suitable for academic evaluation, viva discussion, and portfolio presentation.